

Software Requirements Specification (SRS)

Expense Tracker

Version: 1.0

Project Type: Python Mini Project 3

Frontend: Tkinter (Python GUI)

Backend: Python

Storage: JSON File

Document Control

Prepared by: Md. Anjir Jaman

Date: 05-09-2026

Version: 1.0 – Initial draft

1. Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Intended User

2. Overall Description

- 2.1 Product Perspective
- 2.2 Main Features
- 2.3 Operating Environment

3. Functional Requirements

4. Data Requirements

5. User Interface Requirements

- 5.1 Main Window
- 5.2 Date Picker Dialog
- 5.3 Summary Reports

6. Backend Requirements

7. Data Storage Requirements

8. Non-Functional Requirements

9. Error Handling

10. System Constraints

11. Assumptions

12. Acceptance Criteria

13. Project Schedule and Milestones

14. Requirement Summary

1. Introduction

1.1 Purpose

The Expense Tracker is a desktop Python application, built with Tkinter, that helps a user record and review daily expenses.

The application will provide a simple graphical interface through which users can add expenses, delete expenses, and view summary reports.

Expense records will be stored in a local JSON file so that the data remains available after the application is closed.

1.2 Scope

The system will provide the following main functions:

- Add an expense with an amount, category, and date
- Delete a selected expense
- Pick a date visually using a built-in calendar
- View all recorded expenses in a list
- View total expenses grouped by category
- View total expenses grouped by month
- Validate user input
- Store expense records in a local JSON file
- Load existing expenses when the application starts

The interface will be built using Python's built-in Tkinter library, with no external frontend or web framework.

1.3 Intended User

The system is intended for a user who wants to track personal daily expenses without using dedicated accounting software. The user does not need advanced technical knowledge beyond running a Python script.

2. Overall Description

2.1 Product Perspective

The Expense Tracker will operate as a standalone Python desktop application. The system will use a local JSON file for data storage. No external database, browser, or internet connection is required.

Basic system flow:

User → Tkinter GUI → Python Backend → JSON file (expenses.json)

Figure 1: System Architecture

The Tkinter frontend will collect user input and display information. The Python backend will process requests, perform validation, manage expense records, and handle file operations. The JSON file will provide persistent storage.

2.2 Main Features

- Add Expense
- Delete Expense
- Date Picker
- View Expense List
- Category Totals
- Monthly Totals

2.3 Operating Environment

The application will require:

- A computer
- Python 3
- Tkinter (included with most standard Python installations)
- A Python development environment such as VS Code, PyCharm, or IDLE
- Local file storage

3. Functional Requirements

FR-01: Load Expenses

The system shall load expense records from a local JSON file when the application starts.

If the file does not exist, the system shall start with an empty expense list without displaying an error.

If the file exists but contains invalid JSON, the system shall display an error message and start with an empty list.

FR-02: Add Expense

The system shall provide input fields for Amount, Category, and Date.

The system shall require Amount, Category, and Date to be filled in before adding an expense.

The system shall validate that Amount is a positive number.

The system shall validate that Date matches the YYYY-MM-DD format.

On success, the system shall save the new expense, refresh the displayed list, and clear the input fields.

FR-03: Delete Expense

The system shall allow the user to select an expense from the displayed list.

The system shall delete the selected expense when requested.

If no expense is selected, the system shall display an error message.

FR-04: Date Picker

The system shall provide a button that opens a calendar dialog.

The calendar dialog shall allow the user to navigate between months.

Selecting a day on the calendar shall automatically fill the Date field in YYYY-MM-DD format.

FR-05: View Expense List

The system shall display all recorded expenses in a list, showing the date, amount, and category of each entry.

FR-06: Category Totals

The system shall calculate the total amount spent per category.

The system shall display category totals along with an overall total.

If no expenses are recorded, the system shall display an appropriate message.

FR-07: Monthly Totals

The system shall calculate the total amount spent per month.

The system shall display monthly totals sorted chronologically.

If no expenses are recorded, the system shall display an appropriate message.

FR-08: Save Expenses

The system shall save the complete expense list to the local JSON file after every add or delete operation.

FR-09: Input Validation

The system shall reject empty required fields.

The system shall reject amounts that are not positive numbers.

The system shall reject dates that do not match the YYYY-MM-DD format.

The system shall display clear error messages when invalid information is submitted.

4. Data Requirements

Each expense record shall contain:

Field	Description	Required
Amount	The expense amount (a positive number)	Yes
Category	The category label for the expense (e.g., Food, Transport)	Yes
Date	The date of the expense, in YYYY-MM-DD format	Yes

5. User Interface Requirements

The application shall use a Tkinter-based desktop graphical interface.

5.1 Main Window

The main window shall contain input fields for Amount, Category, and Date, along with the following controls:

- Choose Date button
- Add Expense button
- Delete Expense button
- Category Totals button
- Monthly Totals button
- A list displaying all recorded expenses

5.2 Date Picker Dialog

The dialog shall display a calendar for the currently selected month.

The dialog shall provide controls to move to the previous or next month.

Selecting a day shall close the dialog and fill in the Date field automatically.

5.3 Summary Reports

Category totals and monthly totals shall be displayed in a popup dialog listing each group alongside its total amount.

Example:

Category	Total
Food	\$1,250.00
Transport	\$430.50

6. Backend Requirements

The Python backend shall:

- Load expense records from the storage file at startup
- Validate submitted amount, category, and date values
- Add new expense records
- Delete selected expense records
- Calculate category totals
- Calculate monthly totals
- Read from and write to the storage file after each change
- Return appropriate results and error messages to the interface

7. Data Storage Requirements

The system shall use JSON for local data storage.

Example expenses.json:

```
[{"amount": 250.0, "category": "Food", "date": "2026-09-05"}]
```

The system shall support:

- Creating records
- Reading records
- Deleting records
- Persistent storage across sessions

8. Non-Functional Requirements

NFR-01: Usability

The interface should be simple, clear, and easy to navigate, with clearly labeled fields and buttons.

NFR-02: Performance

Normal operations should complete within a reasonable time for the expected small number of records.

NFR-03: Reliability

The system should correctly save and retrieve expense information during normal operation.

NFR-04: Maintainability

The Python code should be organized clearly, with separate functions for loading, saving, adding, deleting, and reporting, so that the application can be modified or extended easily.

NFR-05: Portability

The application should run on any system with Python 3 and Tkinter installed, without modification.

NFR-06: Data Persistence

Expense records shall remain available after the application is closed and reopened.

9. Error Handling

The system shall handle common errors without crashing.

The system shall provide appropriate messages for:

- Missing required fields (amount, category, or date)
- Invalid amount (non-numeric or not positive)
- Invalid date format
- No expense selected for deletion
- A corrupted or unreadable expenses file

After an error, the user should be able to correct the input and try again without restarting the application.

10. System Constraints

- The application will be developed using Python and Tkinter.
- No external database is required.
- Data will be stored locally using JSON.
- No internet connection is required for normal operation.
- The system is intended for a small number of expense records.
- The project will remain within the scope of a Python mini project.

11. Assumptions

- Python 3 with Tkinter is installed on the development computer.
- The user has permission to read and write files in the project directory.
- The user provides a valid date, either by typing it or using the date picker.
- The storage file is accessible to the application.

12. Acceptance Criteria

The project will be considered complete when:

- The application launches successfully.
- The user can add an expense through the interface.
- The user can delete a selected expense.
- The date picker correctly fills in the Date field.
- Expenses are saved to the JSON file.
- Existing expenses load correctly when the application restarts.
- Category totals display correctly.
- Monthly totals display correctly.
- Invalid input is handled correctly without crashing.
- Appropriate success and error messages are displayed.
- All major functions pass their test cases.

13. Project Schedule and Milestones

The project is scheduled to be completed in a single working session (approximately 6 hours), broken down by task as shown in Figure 2. The schedule follows the SDLC sequence defined in this document: requirements review, design, implementation, testing, and reporting.

Task	9am	10am	11am	12pm	1pm	2pm
Set up project, load/save functions						
Main window + Add Expense (validation)						
Delete expense + list refresh						

Category & Monthly totals reports						
Date picker calendar widget						
Testing + write report						

Figure 2: Development schedule

14. Requirement Summary

FR-01	Load expenses	FR-06	Category totals
FR-02	Add expense	FR-07	Monthly totals
FR-03	Delete expense	FR-08	Save expenses
FR-04	Date picker	FR-09	Validate input
FR-05	View expense list	NFR-01	Usability
NFR-02	Performance	NFR-05	Portability
NFR-03	Reliability	NFR-06	Data persistence
NFR-04	Maintainability		